Composition d'informatique n°2

Proposition de corrigé

⋆ ⋆ ⋆

1 Alphabets, mots et automates

Question 1 Comme il n'y a que deux lettres dans l'alphabet, il suffit d'envisager les quatre cas possibles.

```
let cmp_letter x1 x2 = match x1, x2 with
    | A, B -> -1
    | B, A -> 1
    | _ -> 0
```

Question 2 On compare récursivement lettre à lettre, jusqu'à ce qu'un des deux mots soit vide.

```
let rec cmp_word w1 w2 = match w1, w2 with
    | [], [] -> 0
    | _, [] -> -1
    | [], _ -> 1
    | x1 :: u1, x2 :: u2 ->
        let c = cmp_letter x1 x2 in
        if c = 0 then cmp_word u1 u2
        else c
```

Question 3 Une table de hachage permet d'implémenter un dictionnaire. L'idée est de stocker chaque couple (clé, valeur) dans la case d'un tableau dont l'indice est l'image de la clé par une fonction de hachage. Pour gérer les collisions (clés qui ont la même image), on peut par exemple créer une liste chaînée.

Le module `Hashtbl` dans OCaml implémente les tables de hachage.

Question 4 Il s'agit juste d'une question de syntaxe.

```
let delta_a =
    WordMap.add [] [A] (WordMap.add [A] [] WordMap.empty)

let delta_b =
    WordMap.add [] [] (WordMap.add [A] [A] WordMap.empty)

let un_a =
    WordMap.add [] false (WordMap.add [A] true WordMap.empty)
```

Question 5 Le langage reconnu est l'ensemble des mots sur l'alphabet $\{a, b\}$ ayant un nombre impair de a.

Question 6 En demandant de compléter le code, l'énoncé suggère implicitement d'utiliser la question 4.

```
let automaton_example = {
        initial = [];
        transitions = fun q x
            -> WordMap.find q (if x = A then delta_a else delta_b);
        finals = fun q -> WordMap.find q un_a
    }
```

Question 7 Un classique de programmation sur les automates. Il suffit d'appliquer la formule d'induction $\delta^*(q, xu) = \delta^*(\delta(q, x), u)$.

```
let rec delta_star a q = function
    | [] -> q
    | x :: u -> delta_star a (a.transitions q x) u
```

Question 8 On se contente de calculer $\delta^*(q_0, w)$ et de vérifier s'il est dans F.

```
let accepte a w =
    a.finals (delta_star a a.initial w)
```

2 Arbres bouc-émissaire

Question 9 On compare l'élément cherché à la racine et on cherche à gauche ou à droite en fonction du résultat si les deux sont différents.

```
let rec search x (t, n) = match t with
    | E -> false
    | N (l, y, r) ->
        x = y ||
        (x < y && search x (l, 0)) ||
        (x > y && search x (r, 0))
```

On remarque que le deuxième argument du couple n'est pas utilisé (donc mis à 0 pour les appels récursifs).

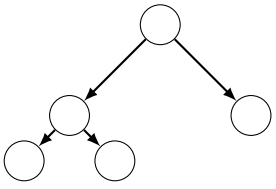
Question 10 Montrons ce résultat par induction.

- si $T = N(E, x, E)$ est une feuille, alors T est à la fois équilibré en taille (car $|\ell| = |r| = 0$) et en hauteur (car $h(T) = 0 \leqslant \log_{1/\alpha}(1)$) ;
- supposons que $T = N(\ell, x, r)$ est équilibré en taille, et que le plus haut enfant (qu'on peut supposer être ℓ sans perte de généralité) de T est non vide et vérifie la propriété (donc est équilibré en taille et en hauteur).

  On a alors :

$$\left(\frac{1}{\alpha}\right)^{h(T)} = \left(\frac{1}{\alpha}\right)^{1+h(\ell)} \leqslant \frac{1}{\alpha}|\ell| \leqslant |T|$$

  L'arbre T est donc équilibré en hauteur et l'induction est vérifiée.

La réciproque est fausse, par exemple avec l'arbre : 

En effet, si on note cet arbre $T = N(\ell, x, r)$, alors $2^{h(T)} = 4 \leqslant 5$, donc T est $\frac{1}{2}$-équilibré en hauteur (les deux enfants étant parfaits). Toutefois, $|\ell| = 3 > \frac{1}{2}|T| = 2,5$, donc T n'est pas $\frac{1}{2}$-équilibré en taille.

Question 11 On calcule un parcours en profondeur infixe, qui correspond exactement au résultat voulu sachant que l'arbre est binaire de recherche. Pour obtenir une complexité linéaire en n, on ne peut pas se permettre de faire le travail en calculant récursivement les parcours des enfants et en les concaténant (sinon la complexité pourrait être quadratique).

On crée un tableau, qu'on remplit à l'aide d'une référence d'indice.

```
let infix (t, n) =
    let tab = Array.make n 0 in
    let i = ref 0 in
    let rec fill = function
        | E -> ()
        | N (l, x, r) ->
            fill l; tab.(!i) <- x; incr i; fill r
    in
    fill t;
    tab
```

Question 12 On suit l'indication (notamment pour éviter des copies de tableaux, coûteuses en temps). Il suffit de couper le tableau en 2 et de recommencer sur chaque moitié avant de reconstruire l'arbre. La complexité suit une formule de récurrence de la forme $C(n) = C\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + C\left(\left\lceil \frac{n-1}{2} \right\rceil\right) + \mathcal{O}(1)$, c'est-à-dire $C(n) = \mathcal{O}(n)$.

```
let balance_array tab =
    let rec balanced_between i j = match j - i with
        | 0 -> E
        | _ ->
            let mid = (i + j) / 2 in
            let l = balanced_between i mid in
            let r = balanced_between (mid + 1) j in
            N (l, tab.(mid), r)
    in
    balanced_between 0 (Array.length tab)
```

Question 13 Pour le cas de base, il suffit de comparer la profondeur au seuil donné par la fonction `threshold` pour savoir si l'arbre est α-déséquilibré en hauteur.

Pour un nœud interne, on fait un appel récursif dans le bon enfant, et on distingue :

- si le booléen vaut `false`, il n'y a pas de rééquilibrage à faire ;
- sinon, on calcule la taille de l'autre enfant, on vérifie si le nœud courant est déséquilibré en taille, et si c'est le cas, on rééquilibre le sous-arbre et le booléen devient `false`. Sinon, on laisse le sous-arbre tel quel et le booléen reste `true`.

On fait attention à renvoyer la bonne valeur pour la taille du sous-arbre si le booléen reste à `true`.

```
    | E -> N (E, x, E), depth > threshold (n + 1), 1
    | N (l, y, r) ->
        if x <= y then
            let l', b, sl = insert_and_balance (depth + 1) l in
            if b then
                let sr = size r in
                if unbalanced sl sr then
                    infix (N (l', y, r), sl + sr + 1), false, 0
                else
                    N (l', y, r), true, sl + sr + 1
            else
                N (l', y, r), false, 0
```

Question 14 Supposons $T = N(\ell, x, r)$ et, sans perte de généralité, que $|\ell| \geqslant |r|$. Alors $|r| \leqslant |\ell| \leqslant \frac{|T|}{2} = \frac{1}{2}(|\ell| + |r| + 1) \leqslant |\ell| + 1$.

On a donc $|\ell| - |r| \leqslant 1$, donc $D(\text{rac}(T)) = 0$.

Question 15 Soit s le nœud bouc-émissaire. Il est donc α-déséquilibré en taille (par définition). Si, sans perte de généralité, $\ell(s)$ est son enfant de plus grande taille, alors $|\ell(s)| > \alpha|T(s)|$. On a alors :

$$D(s) \geqslant |\ell(s)| - |r(s)| - 1 > \alpha|T(s)| - |r(s)| - 1 = (\alpha - 1)|T(s)| + |\ell(s)| > (2\alpha - 1)|T(s)| = \frac{1}{3}|T(s)|$$

Ce qui conclut.

Question 16 Il suffit de remarquer l'apparition d'une somme télescopique. On a la bonne inégalité car $D(T_0) = 0$ et $D(T_k) \geqslant 0$.

Par ailleurs, on remarque que $C(I_j)$ est une bonne estimation de la complexité de l'opération d'insertion I_j, car seul un nombre constant d'opérations sont faites par nœud parcouru. L'inégalité précédente permet de conclure qu'une majoration de $A(I_j)$ est une majoration de la complexité amortie de l'insertion d'un élément dans T_j.

Calculons les valeurs possibles pour $A(I_j)$:

- s'il n'y a pas de rééquilibrage effectué, alors $D(T_j) - D(T_{j-1})$ est inférieur ou égal au nombre de nœuds parcourus (car on fait augmenter la taille d'un des deux enfants de chaque nœud parcouru). Comme T_j est α-équilibré en hauteur, ce nombre est un $\mathcal{O}(\log |T_j|)$;
- s'il y a un rééquilibrage du sous-arbre $T_j(s)$, alors $C(I_j) = \text{prof}(s) + |T_j(s)|$. De plus, $D(T_j) - D(T_{j-1}) \leqslant \text{prof}(s) - D(T'_j(s))$ (où $T'_j(s)$ désigne le sous-arbre enraciné en s avant rééquilibrage), car après rééquilibrage, $D(T_j(s)) = 0$, et seuls les ancêtres de s ont pu avoir leur déséquilibre augmenté de 1.

  En conclusion, $A(I_j) = 4\text{prof}(s) + |T_j(s)| - 3D(T'_j(s)) \leqslant 4\text{prof}(s) = \mathcal{O}(\log |T_j|)$.

Dans tous les cas, $A(I_j) = \mathcal{O}(\log |T_j|)$, ce qui conclut.

3 Relation de séparabilité par rapport à un langage

Question 17 On applique la définition.

```
let separated_by u1 u2 v =
    Oracle.mem (u1 @ v) <> Oracle.mem (u2 @ v)
```

Question 18 Cette question est très facile, mais longue à rédiger. À ce niveau du sujet, on peut se permettre quelques raccourcis.

On remarque que $u_1 \equiv_\Lambda u_2$ si et seulement si pour tout $v \in \Sigma^*$, $\mathbb{1}_\Lambda(u_1 v) = \mathbb{1}_\Lambda(u_2 v)$. Dans cette expression, la relation $=$ est une relation d'équivalence, donc $\equiv_\Lambda$ est une relation d'équivalence (la quantification sur v ne change rien).

Question 19 Soit $v \in \Sigma^*$. Notons $q = \delta^*(q_0, u_1) = \delta^*(q_0, u_2)$. Alors $\mathbb{1}_{L_A}(u_1 v) = \mathbb{1}_F(\delta^*(q, v)) = \mathbb{1}_{L_A}(u_2 v)$. On en déduit que $u_1 \equiv_{L_A} u_2$.

4 Arbre discriminant

Question 20 On remarque que pour le mot $u = ba$, un mot uv ne peut jamais appartenir à $\mathcal{L}(ab^*a)$, car il ne commence pas par a. On en déduit qu'en criblant le mot u à travers $\mathcal{T}_0$, on ira jusqu'à la feuille la plus à droite, donc $\lfloor ba \rfloor = \underline{b}$.

Question 21 On applique, de manière récursive, la définition du calcul du criblat.

```
let rec sift t u = match t with
    | Leaf q -> q
    | Node (l, v, r) ->
        if Oracle.mem (u @ v) then sift l u
        else sift r u
```

Question 22 La complexité temporelle est en $\mathcal{O}(h \times (|u| + \mu(|u| + M)))$. En effet, on constate qu'à chaque appel récursif, la hauteur de l'arbre donné en argument diminue de 1. Il y a donc au plus h appels récursifs. De plus, chaque appel récursif nécessite de :

- calculer `u @ v`, ce qui se fait en temps $\mathcal{O}(|u|)$;
- faire l'appel à l'oracle sur uv, ce qui se fait en temps $\mathcal{O}(\mu(|u| + M))$ (en supposant que μ est une fonction croissante).

Question 23 Soient $u_1, u_2 \in \Sigma^*$ tels que $\lfloor u_1 \rfloor \neq \lfloor u_2 \rfloor$. Notons $s_0 - s_1 - \cdots - s_k$ et $t_0 - t_1 - \cdots - t_\ell$ les chemins obtenus en criblant u_1 et u_2 respectivement.

Sachant que $s_0 = t_0$ et que $s_k \neq t_\ell$, il existe un indice $i \geqslant 0$ tel que $s_i = t_i$ et $s_{i+1} \neq t_{i+1}$. Posons v_i le mot au nœud $s_i = t_i$.

Sachant que $s_{i+1} \neq t_{i+1}$, on en déduit que u_1 et u_2 sont séparés par v_i, donc $u_1 \not\equiv_L u_2$.

Question 24 D'après la question 23, le nombre de classes d'équivalences d'inséparabilité est fini, majoré par le nombre d'états n d'un automate qui reconnaît L.

Soit T un arbre discriminant accessible. Par l'absurde, supposons que T possède strictement plus que n feuilles.

Par le principe des tiroirs, il existe donc deux feuilles distinctes $\underline{w_1}$ et $\underline{w_2}$ telles que $w_1 \equiv_L w_2$. Par la question précédente et par accessibilité :

$$\underline{w_1} = \lfloor w_1 \rfloor = \lfloor w_2 \rfloor = \underline{w_2}$$

ce qui est absurde, car on les a supposées distinctes.

On en déduit que le nombre de feuilles de T est majoré par n.

Question 25 Si $\lfloor u_1 \rfloor = \lfloor u_2 \rfloor$ par rapport à un arbre discriminant démêlant T, alors en particulier, le premier test réalisé lors du crible de u_1 et u_2 à travers T doit être identique, donc $\mathbb{1}_L(u_1\varepsilon) = \mathbb{1}_L(u_2\varepsilon)$.

5 Automate tiré d'un arbre discriminant

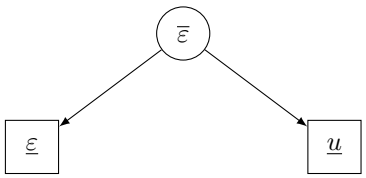
Question 26 On applique la définition. Comme les états finals sont ceux à gauche de la racine, ce sont ceux qui appartiennent au langage.

```
let automaton_of_disctree t = {
        initial = [];
        transitions = fun q x -> sift t (q @ [x]);
        finals = Oracle.mem
    }
```

Question 27 On commence par faire un appel à `Oracle.mem []` et on distingue deux cas :

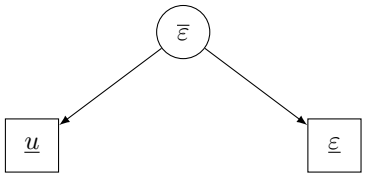
- si $\varepsilon \in L$, alors il existe $u \in \Sigma^*$ tel que $u \notin L$ (car $L \neq \Sigma^*$). Un tel mot u peut être obtenu en faisant un appel à `Oracle.equiv a`, où `a` est un automate à un seul état qui est initial et final, avec les transitions qui bouclent sur lui-même (c'est-à-dire reconnaissant Σ^*).

On construit alors le crible :



– si $\varepsilon \notin L$, alors il existe $u \in \Sigma^*$ tel que $u \in L$ (car $L \neq \emptyset$). Un tel mot u peut être obtenu en faisant un appel à `Oracle.equiv a`, où `a` est un automate à un seul état qui est initial **mais non final**, avec les transitions qui bouclent sur lui-même (c'est-à-dire reconnaissant $\emptyset$).

On construit alors le crible :



Question 28 On a $\tau_\gamma = p_\gamma = \lfloor c \rfloor$ et $\hat{\tau}_\gamma = \hat{p}_\gamma = \delta^*(\underline{\varepsilon}, c)$.

Distinguons :

– si $c \in L$, alors l'automate associé à T ne reconnaît pas c (car c est un contre-exemple), et donc $\delta^*(\underline{\varepsilon}, c)$ est une feuille dans l'enfant droit de T. Par ailleurs, comme $c \in L$, lorsqu'on crible c, on commence par le côté gauche. On en déduit que $\lfloor c \rfloor$ est une feuille dans l'enfant gauche.

– on raisonne symétriquement si $c \notin L$.

Dans les deux cas, l'un est gauche et l'autre à droite. Par ailleurs, comme T est accessible, pour chaque feuille w, $\lfloor w \rfloor = \underline{w}$. Comme T est démêlant, le premier test d'appartenance obtenu en criblant un mot w est le test $w \in L$. On en déduit qu'entre $\tau_\gamma = p_\gamma$ et $\hat{\tau}_\gamma$, l'un est dans L (celui à gauche) et pas l'autre.

Question 29 Il y a peut-être une manière efficace de faire le calcul, mais comme aucune complexité n'est imposée, on se propose d'appliquer la définition pour les calculs des différents mots. De toute façon, refaire le découpage à chaque fois a la même complexité que calculer les criblats ou les images par δ^*.

On commence par écrire une première fonction `cut` telle que `cut i c` renvoie le triplet (π_i, c_i, σ_i).

```
let rec cut i c = match i, c with
    | 0, c_0 :: c' -> [], c_0, c'
    | gamma, x :: c' ->
        let pi_i, c_i, sigma_i = cut (i - 1) c' in
        x :: pi_i, c_i, sigma_i
    | _ -> failwith "Erreur"
```

Ensuite, on peut appliquer la définition pour trouver j, en utilisant les fonctions écrites précédemment.

```
let split t c =
    let a = automaton_of_disctree t in
    let i = ref 0 and trouve = ref false in
    while not !trouve do
        let pi_i, c_i, sigma_i = cut !i c in
        let p_i = sift t pi_i and
            hat_p_i = delta_star a [] pi_i in
        if cmp_word p_i hat_p_i <> 0 then trouve := true
        else incr i
    done;
    let pi_j, c_j, sigma_j = cut !i c in
    let p_j = sift t pi_j and p_j_plus_un = sift t (pi_j @ [c_j]) in
    p_j @ [c_j], p_j_plus_un, sigma_j
```

Question 30 Il y a deux choses à faire ici : trouver le nœud $\underline{p_{j+1}}$ à remplacer, et déterminer l'ordre des enfants. Pour le deuxième point, il suffit de tester si $p_{j+1}\sigma_j \in L$ (auquel cas $\underline{p_{j+1}}$ est à gauche) ou non (auquel cas il est à droite).

Pour le premier point, on procède comme pour le crible, car $\underline{p_{j+1}} = \lfloor p_{j+1} \rfloor$.

```
let rec substitute t (pp, p, s) = match t with
    | Leaf v ->
        if cmp_word v p = 0 then
            if Oracle.mem (p @ s) then
                Node (Leaf p, s, Leaf pp)
            else
                Node (Leaf pp, s, Leaf p)
        else Leaf v
    | Node (l, v, r) ->
        if Oracle.mem (p @ v) then
            substitute l (pp, p, s)
        else
            substitute r (pp, p, s)
```

Question 31 Comme aucune spécification de fonction n'est indiquée, on suppose qu'on attend ici une description de haut niveau en français par exemple. On propose l'algorithme suivant :

- Initialiser un crible T avec la question 27.
- Tant que l'automate associé à T ne reconnaît pas L :
  - Soit c un contre-exemple renvoyé par l'oracle.
  - Calculer le triplet $(p_j c_j, p_{j+1}, \sigma_j)$ à l'aide de la question 29.
  - Remplacer T par l'arbre obtenu à l'aide de la question 30.
- Renvoyer l'automate associé à T.

On peut rajouter des premiers tests d'équivalence avec un automate à un seul état pour vérifier si $L = \emptyset$ ou $L = \Sigma^*$.

Le nombre de feuilles de T est un variant strictement croissant, et on a montré à la question 24 qu'il était borné, donc l'algorithme termine.

Question 32 En question 24, on a montré que le nombre de feuilles d'un crible est majoré par le nombre d'état de tout automate reconnaissant L. Par ailleurs, l'automate associé au crible de la question précédente contient autant d'états que de feuilles du crible. On en déduit que ce nombre d'état est minimal.